99. Передача по адресу

Есть еще один способ передачи переменных в функцию в языке C++ — по адресу.

Оглавление:

1. Передача по адресу
2. Передача по константному адресу
3. Передача адресов по ссылке
4. Существует только передача по значению
5. Плюсы и минусы передачи по адресу

Передача по адресу

Передача аргументов по адресу — это передача адреса переменной-аргумента (а не значения исходной переменной). Поскольку аргумент является адресом, то параметром функции должен быть **указатель**. Затем функция сможет разыменовать этот указатель для доступа или изменения исходного значения. Вот пример функции, которая принимает параметр, передаваемый по адресу:

```
#include <iostream>

void boo(int *ptr)
{
    *ptr = 7;
}

int main()
{
    int value = 4;

    std::cout << "value = " << value << "\n";
    boo(&value);
    std::cout << "value = " << value << "\n";
    return 0;
}
```

Результат выполнения программы:

```
value = 4
value = 7
```

Как вы можете видеть, функция boo() изменила значение аргумента (переменную `value`) через параметр-указатель `ptr`. Передачу по адресу обычно используют с указателями на обычные **массивы**. Например, следующая функция выведет все значения массива:

```
void printArray(int *array, int length)
{
    for (int index=0; index < length; ++index)
        std::cout << array[index] << " ";
}
```

Вот пример программы, которая вызывает эту функцию:

```
int main()
{
    int array[7] = { 9, 8, 6, 4, 3, 2, 1 }; // помните, что массивы распадаются в указатели при передаче
    printArray(array, 7); // передаем адрес array + его указатель на первый элемент массива (с использованием оператора & нет нес
}
```

Результат:

```
9 8 6 4 3 2 1
```

Помните, что **фиксированные массивы распадаются в указатели** при передаче в функцию, поэтому их длину нужно передавать в виде отдельного параметра. Перед разыменованием параметров, передаваемых по адресу, не лишним будет проверить — не являются ли они **нулевыми указателями**. Разыменование нулевого указателя приведет к сбою в программе. Вот функция printArray() с проверкой (обнаружением) нулевых указателей:

```
#include <iostream>

void printArray(int *array, int length)
{
    // Если пользователь передал нулевой указатель в качестве array
    if (!array)
        return;

    for (int index=0; index < length; ++index)
        std::cout << array[index] << " ";
}

int main()
{
    int array[7] = { 9, 8, 6, 4, 3, 2, 1 };
    printArray(array, 7);
}
```

Передача по константному адресу

Поскольку printArray() все равно не изменяет значения получаемых аргументов, то хорошей идеей будет сделать параметр `array` константным:

```
#include <iostream>

void printArray(const int *array, int length)
{
    // Если пользователь передал нулевой указатель в качестве array
    if (!array)
        return;

    for (int index=0; index < length; ++index)
        std::cout << array[index] << " ";
}

int main()
{
    int array[7] = { 9, 8, 6, 4, 3, 2, 1 };
    printArray(array, 7);
}
```

Так мы видим сразу, что printArray() не изменит переданный аргумент `array`. Когда вы передаете указатель в функцию по адресу, то значение этого указателя (адрес, на который он указывает) копируется из аргумента в параметр функции. Другими словами, он **передается по значению**! Если изменить значение параметра функции, то изменится только копия, исходный указатель-аргумент не будет изменен. Например:

```
#include <iostream>

void setToNull(int *tempPtr)
{
    // Мы присваиваем tempPtr другое значение (мы не изменяем значение, на которое указывает tempPtr)
    tempPtr = nullptr; // используйте 0, если не поддерживается C++11
}

int main()
{
    // Сначала мы присваиваем ptr адрес six, т.е. *ptr = 6
    int six = 6;
    int *ptr = &six;

    // Здесь выводится 6
    std::cout << *ptr << "\n";

    // tempPtr получает копию ptr
    setToNull(ptr);

    // ptr по-прежнему указывает на переменную six!

    // Здесь выводится 6
    if (ptr)
        std::cout << *ptr << "\n";
    else
        std::cout << " ptr is null";

    return 0;
}
```

В `tempPtr` копируется адрес указателя `ptr`. Несмотря на то, что мы изменили `tempPtr` на нулевой указатель (присвоили ему `nullptr`), это никак не повлияло на значение, на которое указывает `ptr`. Следовательно, результат выполнения программы:

```
6
6
```

Обратите внимание, хотя сам адрес передается по значению, вы все равно можете разыменовать его для изменения значения исходного аргумента. Запутано? Давайте поясним:

→ При передаче аргумента по адресу в переменную-параметр функции копируется адрес из аргумента. В этот момент параметр функции и аргумент указывают на одно и то же значение.

→ Если параметр функции затем разыменовать для изменения исходного значения, то это приведет к изменению значения, на которое указывает аргумент, поскольку параметр функции и аргумент указывают на одно и то же значение!

→ Если параметру функции присвоить другой адрес, то это никак не повлияет на аргумент, поскольку параметр функции является копией, а изменение копии не приводит к изменению оригинала. После изменения адреса параметра функции, параметр функции и аргумент будут указывать на разные значения, поэтому разыменование параметра и дальнейшее его изменение никак не повлияют на значение, на которое указывает аргумент.

В следующей программе это всё хорошо проиллюстрировано:

```
#include <iostream>

void setToSeven(int *tempPtr)
{
    *tempPtr = 7; // мы изменяем значение, на которое указывает tempPtr (= ptr тоже)
}

int main()
{
    // Сначала мы присваиваем ptr адрес six, т.е. *ptr = 6
    int six = 6;
    int *ptr = &six;

    // Здесь выводится 6
    std::cout << *ptr << "\n";

    // tempPtr получает копию ptr
    setToSeven(ptr);

    // tempPtr изменил значение, на которое указывал, на 7

    // Здесь выводится 7
    if (ptr)
        std::cout << *ptr << "\n";
    else
        std::cout << " ptr is null";

    return 0;
}
```

Результат выполнения программы:

```
6
7
```

Передача адресов по ссылке

Следует вопрос: «А что, если мы хотим изменить адрес, на который указывает аргумент, внутри функции?». Оказывается, это можно сделать очень легко. Вы можете просто передать адрес по ссылке. Синтаксис ссылки на указатель может показаться немного странным, но все же:

```
#include <iostream>

// tempPtr теперь является ссылкой на указатель, поэтому любые изменения tempPtr приведут к к изменению исходного аргумента!
void setToNull(int *&tempPtr)
{
    tempPtr = nullptr; // используйте 0, если не поддерживается C++11
}

int main()
{
    // Сначала мы присваиваем ptr адрес six, т.е. *ptr = 6
    int six = 6;
    int *ptr = &six;

    // Здесь выводится 6
    std::cout << *ptr;

    // tempPtr теперь ссылается на ptr
    setToNull(ptr);

    // ptr было присвоено значение nullptr!

    if (ptr)
        std::cout << *ptr;
    else
        std::cout << " ptr is null";

    return 0;
}
```

Результат выполнения программы:

```
6 ptr is null
```

Наконец, наша функция setToNull() действительно изменила значение `ptr` с `&six` на `nullptr`!

Существует только передача по значению

Теперь, когда вы понимаете основные различия между передачей по ссылке, по адресу и по значению, давайте немного поговорим о том, что находится «под капотом».

На уроке о **ссылках** мы упоминали, что ссылки на самом деле реализуются с помощью указателей. Это означает, что передача по ссылке является просто передачей по адресу. И чуть выше мы говорили, что передача по адресу на самом деле является передачей адреса по значению! Из этого следует, что C++ действительно передает всё по значению!

Плюсы и минусы передачи по адресу

Плюсы передачи по адресу:

→ Передача по адресу позволяет функции изменить значение аргумента, что иногда полезно. В противном случае, используем const для гарантии того, что функция не изменит аргумент.

→ Поскольку копирования аргументов не происходит, то скорость передачи по адресу достаточно высокая, даже если передавать большие **структуры** или классы.

→ Мы можем вернуть сразу несколько значений из функции, используя **параметры вывода**.

Минусы передачи по адресу:

→ Все указатели нужно проверять, не являются ли они нулевыми. Попытка разыменовать нулевой указатель приведет к сбою в программе.

→ Поскольку разыменование указателя выполняется медленнее, чем доступ к значению напрямую, то доступ к аргументам, переданным по адресу, выполняется также медленнее, чем доступ к аргументам, переданным по значению.

Когда использовать передачу по адресу:

→ при передаче обычных массивов (если нет никаких проблем с тем, что массивы распадаются в указатели при передаче).

Когда не использовать передачу по адресу:

→ при передаче структур или классов (используйте передачу по ссылке);

→ при передаче фундаментальных типов данных (используйте передачу по значению).

Как вы можете видеть сами, передача по адресу и по ссылке имеют почти одинаковые преимущества и недостатки. Поскольку передача по ссылке обычно безопаснее, чем передача по адресу, то в большинстве случаев предпочтительнее использовать передачу по ссылке.

Правило: Используйте передачу по ссылке, вместо передачи по адресу, когда это возможно.